

---

# **sd-boot Documentation**

*Release 6.5.7*

**Gene C**

**Oct 03, 2026**



# SD-BOOT DOCUMENTATION

|           |                                             |           |
|-----------|---------------------------------------------|-----------|
| <b>1</b>  | <b>sd-boot</b>                              | <b>1</b>  |
| 1.1       | Overview . . . . .                          | 1         |
| 1.2       | Quick Start . . . . .                       | 3         |
| <b>2</b>  | <b>Configuration</b>                        | <b>5</b>  |
| 2.1       | Getting Started . . . . .                   | 6         |
| 2.2       | Command Line Tools . . . . .                | 6         |
| <b>3</b>  | <b>Installing Arch Kernels With sd-boot</b> | <b>9</b>  |
| <b>4</b>  | <b>Boot Loader Entries</b>                  | <b>11</b> |
| <b>5</b>  | <b>Layout: BLS vs UKI</b>                   | <b>13</b> |
| <b>6</b>  | <b>Signing Kernels &amp; Secure Boot</b>    | <b>15</b> |
| 6.1       | Overview . . . . .                          | 15        |
| 6.2       | What To Do . . . . .                        | 16        |
| <b>7</b>  | <b>Dracut options</b>                       | <b>21</b> |
| <b>8</b>  | <b>Efi Filesystem Drivers</b>               | <b>23</b> |
| <b>9</b>  | <b>Skipping Kernel Plugins</b>              | <b>25</b> |
| <b>10</b> | <b>Bootable efi tools</b>                   | <b>27</b> |
| 10.1      | A note on memtest86 plus . . . . .          | 28        |
| <b>11</b> | <b>Development</b>                          | <b>29</b> |
| 11.1      | C-Code Language Choice . . . . .            | 29        |
| 11.2      | Testing C-Code . . . . .                    | 29        |
| 11.3      | Development and Debugging . . . . .         | 30        |
| <b>12</b> | <b>Appendix</b>                             | <b>31</b> |
| 12.1      | Compile and Loader Options . . . . .        | 31        |
| 12.2      | Potential Todo Items . . . . .              | 32        |
| <b>13</b> | <b>Man Pages</b>                            | <b>33</b> |

|      |                                    |    |
|------|------------------------------------|----|
| 13.1 | sd-boot-efifs-update . . . . .     | 33 |
| 13.2 | sd-boot-efi-tool-update . . . . .  | 34 |
| 13.3 | sd-boot-find-boot-mounts . . . . . | 35 |
| 13.4 | sd-boot-kernel-update . . . . .    | 38 |

## SD-BOOT

## 1.1 Overview

**sd-boot** installs linux kernels and efi programs to the *boot* partition using *systemd's kernel-install*. This is a robust and systematic way to install or remove bootable software to or from the *boot* partition.

*kernel-install* uses *\$BOOT* to refer to this partition or it's mount point. It is usually mounted on either */efi* or */boot*. In the past it */boot/efi* has also been used, but mounting the ESP under */boot* is now strongly discouraged.

The boot process requires access to the ESP partition and there are two distinct recommendations on mount points.

Mount the ESP partition on:

- */efi* and an XBOOTLDR partition, if any, onto */boot*
- */boot* or on */efi* if there is an XBOOTLDR partition and mount that on */boot*.

The first is recommended by *kernel-install* and the second by the [UAPI Group Specifications: Mount Points](#).

*sd-boot* and *kernel-install* both happily work with either approach.

Our preference follows *kernel-install*; mount the ESP on */efi* and an XBOOTLDR partition on */boot*. This is clean, simple and unambiguous.

We designate the ESP mount point location as *<EFI>*, which is either */efi* or */boot* as appropriate. We also refer to the directory where kernels are put as *<BOOT>*. If you mount the ESP onto */boot* then both *<EFI>* and *<BOOT>* refer to */boot*. If the ESP is mounted on */efi* and you have an XBOOTLDR partition mounted on */boot* then they refer to those (obviously).

While we recommend using the first approach for these mount points, we must also support any systems where the ESP is mounted on */boot*. This guarantees that all systems continue to work without any change.

*sd-boot* provides:

- *pacman alpm hooks*  
These ensure that kernels and efi-tools are installed or removed from the boot partition when a kernel package is installed, updated or removed.

- installers that are triggered from ALPM hooks.  
These do the actual work.
- command line tools  
Give administrators a simple way to directly (re-)install or remove kernels from the boot partition.
- kernel-install plugin(s)  
Used for loader entries for non-UKI kernels and efi tools.

*Installing* a kernel (or efi tool) means placing the appropriate image in *\$BOOT* partition so that the image can be booted.

Any kernel packages must therefore place kernel images in the */usr* partition only.

The same applies for any bootable efi tools such as *edk2-shell* and *memtest86+*.

### 1.1.1 Documentation

The manual provides detailed information about using sd-boot. It is available in both HTML and PDF formats. Both are installed under */usr/share/sd-boot/docs*

The manual is also available at: [readthedocs](#).

Each executable comes with a man page:

- sd-boot-efifs-update
- sd-boot-efi-tool-update
- sd-boot-find-boot-mounts
- sd-boot-kernel-update

For convenience, the manual includes all the man pages.

### 1.1.2 Coding Standards

All code should be carefully reviewed and checked. This is particularly true for system utilities where the importance of avoiding potential problems is high.

Placing trust in software requires care to ensure that it not only performs the required tasks but that the code is robust, handles exceptions gracefully and does not suffer from potential memory related issues. And of course no bugs! Sure, having zero bugs ever is tricky. While we cannot guarantee perfection, there are things we can do to minimize them and bolster confidence in the code.

To help with this, sd-boot has undergone:

- functional testing
- static analysis with tools such as clang-tidy
- compiler analysis with -fanalyzer and -fsanitize
- running the test suite under valgrind and all are completely clean
- human code review
- additional AI reviews.
  - Assisted-by: Claude (Anthropic) <<https://claude.ai>>

The shared library only contains the necessary exported symbol names and all code is compiled and linked with options that enhance safety and security of the compiled code.

Additional details on compile and link options are given in the [Compile and Loader Options](#) of the Appendix.

## 1.2 Quick Start

There are two sets of configuration files

- kernel-install configuration
- sd-boot configuration

Both follow the Linux standard priority order and *drop-in* files that override settings in lower priority files. In rough terms:

- Files within a directory are prioritized alpha-numerically.
- Files in */etc* override those in */usr*
- System provided files reside in */usr*
- Administration settings reside in */etc*
- Drop-in files in directory *xxx.d* may override partial settings.

See *man 5 systemd.unit* for additional information about drop-in configuration files.

### 1.2.1 kernel-install

Please see the kernel-install documentation for full details. sd-boot provides:

- */usr/lib/kernel/install.conf.d/010-sd-boot-install.conf*

which sets the following:

```
layout=uki
initrd_generator=dracut
uki_generator=ukify
```

These may be overridden in:

- */etc/kernel/install.conf*
- */etc/kernel/install.conf.d/xxx.conf*

We recommend sticking with UKI and dracut.

### 1.2.2 sd-boot

All the configuration files are located in */etc/sd-boot*:

- ***/etc/sd-boot/config.yaml***  
Settings:

```
verb: 1
skip_kernel_plugins: []
```

- **/etc/sd-boot/efi-tool.packages**

A list of all package names of efi tools to be managed by sd-boot:

```
memtest86_64-git
edk2-shell
```

- **/etc/sd-boot/kernel.packages**

A list of kernel packages managed by sd-boot:

```
linux-custom
linux-next
linux-test
linux-stable
```

Unlike kernels which are all installed in a fixed location, efi tools must provide an *<package-name>.image* file which has the full path to the *.efi* program.

These are included in sd-boot package:

- `/etc/sd-boot/edk2-shell.image`
- `/etc/sd-boot/memtest86_64-git.image`

Comment lines starting with **#** are permitted in all config files.

### 1.2.3 dracut

sd-boot provides a default dracut configuration in `/usr/lib/dracut/dracut.conf.d/010-sd-boot-dracut.conf` which may be overridden in:

- **`/etc/dracut.conf.d/010-sd-boot-dracut.conf`**

---

## CONFIGURATION

In addition to the files `kernel-install` itself uses, such as those in the `/etc/kernel/` directory, the `sd-boot` configuration files reside in `/etc/sd-boot/`.

The file `/etc/sd-boot/kernel.packages` lists the kernel package names `sd-boot` is permitted to install.

By default, the Arch kernel is not included in the list, but of course `sd-boot` can manage that too. The section [Installing Arch Kernels With sd-boot](#) explains how to use `sd-boot` for the Arch kernel.

Similarly, `/etc/sd-boot/efi-tools.packages` lists the efi tool packages that `sd-boot` is permitted to install.

Package names here are the standard Arch package names.

Please ensure any package to be managed by `sd-boot` is listed appropriately.

By default `kernel-install` sets kernel boot options using in order:

```
/etc/kernel/cmdline  
/usr/lib/kernel/cmdline  
/proc/cmdline
```

Putting kernel command line options into `/etc/kernel/cmdline` will then override the default. If no file provides the kernel options, then The default is to use `/proc/cmdline` which provides the kernel command line option of the currently booted kernel.

The `sd-boot` package provides defaults for `kernel-install` in:

```
/usr/lib/kernel/install.conf.d/010-sd-boot-install.conf
```

which sets the layout to `uki`, the `initrd` generator to `dracut` and the `uki` generator to `ukify`.

Any settings in `/etc/kernel/install.conf` will override the defaults.

The [RFC 66 Discussion](#) about changing to use `kernel-install` inspired us to migrate our kernels and see how it works in practice. We found it works really well and are now sharing this with the community in case its helpful to others.

The first version of `sd-boot` was written in `bash`, but this was replaced by `C` code soon after and is what is used in the current version.

## 2.1 Getting Started

For a kernel to be managed by *sd-boot* the kernel package should install to:

```
/usr/lib/modules/<kernel-version>
```

with the kernel image at:

```
/usr/lib/modules/<kernel-version>/vmlinuz
```

A kernel package must **NOT** install the kernel or any initrd into *<EFI>* or *<BOOT>*. It should not even create an initrd. Installing anything to these *\$BOOT* directories must be left to *sd-boot*. This applies equally to any efi tools.

Once the package is installed, then grant *sd-boot* permission to manage those kernels and efi-tools by listing them in the files:

```
/etc/sd-boot/kernel.packages  
/etc/sd-boot/efi-tools.packages
```

On the next update (or re-install or remove), if the kernel or efi-tool is designated to be managed by *sd-boot* then it will handle installing it into *\$BOOT*.

You can (re-)install a kernel at any time at the command line using:

```
sd-boot-kernel-install <kernel package name>
```

or trigger a pacman refresh with :

```
touch /usr/lib/modules/<kernel-version>/vmlinuz  
pacman -Syu
```

*sd-boot* relies on each kernel package providing a file in the kernel module directory that contains that kernel package name:

```
/usr/lib/modules/<kernel-version>/pkgbase
```

or:

```
/usr/lib/modules/<kernel-version>/pkgbase-sdb
```

Arch kernels provide *pkgbase* file. While *sd-boot* uses either of these files, *pkgbase-sdb* is preferred since the Arch kernel install tools act on any kernel using *pkgbase*. Using a different filename prevents the Arch tools from installing kernels *sd-boot* is already handling.

## 2.2 Command Line Tools

The following tools may be run from the command line:

- `/usr/bin/sd-boot-efi-tool-update`
- `/usr/bin/sd-boot-kernel-update`
- `/usr/lib/sd-boot/sd-boot-efifs-update`

- `/usr/lib/sd-boot/sd-boot-find-boot-mounts`

There is a man page for each of these. For example see *man sd-boot-kernel-update*.

sd-boot is normally triggered into action from pacman via ALPM hooks. Kernels and efi tools may also be manually installed or removed from \$BOOT.

For example to install a bootable efi shell, provided by the *adk2-shell* package:

```
sd-boot-efi-tool-update add edk2-shell
```

To install a kernel provided by the *linux-custom* package

```
sd-boot-kernel-update add linux-custom
```

Please ensure that any packages to be managed by sd-boot are listed appropriately in:

```
/etc/sd-boot/efi-tool.packages  
/etc/sd-boot/kernel.packages
```

To install efi filesystem drivers:

```
/usr/lib/sd-boot/sd-boot-efifs-update add
```

To list ESP and XBOOTLDR partions mount points run (as root):

```
/usr/lib/sd-boot/sd-boot-find-boot-mounts
```

which will list them all and those in use by the current booted system are marked with an asterisk.

Replace *add* by *remove* to remove them from \$BOOT. Note that *add* and *remove* refers only to a copy of the image in \$BOOT and does not install or remove the package itself. That job belongs to pacman.



## INSTALLING ARCH KERNELS WITH SD-BOOT

By default the Archlinux kernel package, *linux*, is not in the list of kernels managed by sd-boot.

In order for sd-boot to manage it, there are two steps that must be done.

- Add **linux** to the list of sd-boot managed kernels:

```
echo linux >> /etc/sd-boot/kernel.packages
```

- Prevent mkinitcpio from installing a duplicate of the same kernel:

```
sudo mkdir -p /etc/pacman.d/hooks  
sudo ln -s /dev/null /etc/pacman.d/hooks/60-mkinitcpio-remove.hook  
sudo ln -s /dev/null /etc/pacman.d/hooks/90-mkinitcpio-install.hook
```



## BOOT LOADER ENTRIES

There are two kinds of systemd boot loader entries that show up in the boot menu.

- **type #1**
  - Uses a separate *loader entry file*
  - Used when there are separate kernel and initrd images or for efi tools.
- **type #2**
  - Used with UKI images only.  
UKI images have the kernel and initrd combined into a single file where the UKI file itself is sufficient to provide the boot loader menu without any additional loader entry file.

Since sd-boot uses *kernel-install* to do the real work, type #1 boot loader entries are automatically created with *bls* layout. It creates one new entry for each kernel version.

After using sd-boot the very first time (only), you should remove any stale (fixed) loader entries from */boot/loader/entries* or */efi/loader/entries* (if that's where they are) for any kernel package now managed by sd-boot.

sd-boot also provides a kernel-install plugin that modifies the raw type#1 loader entries. These are generated for efi tools and for kernels using *bls* layout.

The primary difference is the *Title* which is modified to be the package name. The title is shown in the boot menu.

By default the title is *Arch Linux* which comes from */etc/os-release* for every kernel and efi tool.

efi-tools additionally remove the kernel boot command line options from the entry (since they are irrelevant) and change:

```
linux <tool>.efi
```

to:

```
efi <tool>.efi
```

It also means that the systemd-boot *loader.conf* file located in the *EFI* needs to be updated as well when the default kernel to boot is one of those managed by sd-boot.

Lets show how we can use this so that a *linux-custom* kernel is booted by default.

The loader entries are located in:

```
<EFI>/loader/entries/<machine-id>-<kernel-version>.conf
```

for example:

```
<EFI>/loader/entries/<machine-id>-7.2.0-custom-1.conf
```

We would match this entry in *loader.conf* to be the default kernel using:

```
# loader.conf
default *-custom-*
timeout 5
editor yes
```

The wildcards are standard shell file globbing used to match the loader entry filename. After making this change, its helpful to verify everything is correct by running:

```
bootctl
bootctl list
```

For *uki* layout there are no loader entry files but the comments on *loader.conf* apply similarly. The *loader.conf* example should work fine for BLS and UKI layouts.

## LAYOUT: BLS VS UKI

First off, we prefer the *uki* layout. It is simpler, does not require separate loader entry files, and both kernel and initrd are signed since they are in one file. The *uki* file itself is an *efi* file and can therefore be directly booted without the need of a boot manager should that ever be needed.

For these reasons the default *install.conf* file provided by *sd-boot* uses *uki* layout.

With UKI layout kernel image is written to:

```
$BOOT/EFI/Linux/<machine-id>-<kernel-version>.efi
```

Of particular note is that there are no loader entry files with UKI layout.

BLS layout uses three separate files:

```
$BOOT/<machine-id>/<kernel-version>/linux  
$BOOT/<machine-id>/<kernel-version>/initrd  
$BOOT/loader/entries/<<machine-id>-<kernel-version>.conf
```

kernel-install uses settings taken from */usr/lib/kernel* that may be overridden with files in */etc/kernel*.

sd-boot provides:

```
/usr/lib/kernel/install.conf.d/010-sd-boot-install.conf
```

which contains:

```
layout=uki  
initrd_generator=dracut  
uki_generator=ukify
```

To switch back to BLS from UKI layout create or edit the file:

```
/etc/kernel/install.conf
```

and set *layout=bls*. Then install the kernel package again.

BLS layout uses *type #1* loader entries. If layout is changed from *bls* back to *uki* neither the old BLS loader entry files nor the BLS kernel and initrd are automatically removed. These need to be manually removed.

```
rm $BOOT/loader/entries/<<machine-id>-<kernel-version>.conf  
rm -rf $BOOT/<machine-id>/<kernel-version>
```

While dracut can generate the UKI file without using ukify, this has some limitations. As of now, I recommend using ukify.

We would like to provide an additional option *OSRelease=* to ukify with a modified version of os-release having *PRETTY\_NAME* and *BUILD\_ID* leading to a more user friendly boot menu title. This option is available in the ukify command line tool, but kernel-install does not call this, instead it imports the ukify python module and calls the functions directly. The kernel-install code does not provide for the *OSRelease=* option.

At this time, however, there is no clean way to do this that I could find. Hopefully kernel-install will allow this in the future. In the meantime the boot menu items in *uki* mode are precise but quite long.

## SIGNING KERNELS & SECURE BOOT

Kernels will be signed if keys are available thanks to `kernel-install` and `sbctl`. Many thanks to Foxboron for writing `sbctl` which simplifies things enormously.

This is a very brief overview. Please see Arch wiki and man pages for details on secure boot and kernel signing.

### 6.1 Overview

This note is purposefully brief and incomplete but provides some background around this somewhat large and complex topic. For simplicity and because it is the current desirable approach I will focus on UKI and not BLS layout here.

Signing a kernel provides a way to be sure that it has not changed after the signature was created. Signatures are stored to the TPM hardware from where they can be read during the boot process. There are several slots available and specific slots are designated for specific purposes.

To skip to the section on configuring `systemd-tpm2-setup`, see: [Systemd-TPM2-Setup](#).

#### 6.1.1 Secure Boot On

PCR slot 7 holds the UEFI secure boot state along with the valid certificate database, *db*. It also holds an ever growing list of invalid certificates, *dbx*. Some machines may even no longer have enough space to hold *dbx*.

PCR 7 is required by secure boot. Only a UKI that can be validated by the certificates in slot 7 are permitted to boot by the UEFI firmware.

That is all that is required for secure boot.

#### 6.1.2 Systemd-TPM2-Setup

Systemd provides additional useful checks on the operating system via `systemd-tpm2-setup-early` and `systemd-tpm2-setup` services. This is most useful when secure boot is disabled. It's pretty benign and redundant when using secure boot.

To take advantage of these, *ukify* is used to provide keys and sign the hashes of the `initrd`, the kernel image, the kernel cmdline, any embedded OS-release metadata, splash screen image and, importantly, the `systemd-stub`.

At boot time systemd-stub, computes the hash of the same set of individual hashes and pushes it down to PCR-11. A little later in the boot cycle, systemd extracts the public key from the UKI along with the signed hash. It then uses the public key to verify the signature is authentic and that the hash inside the signature matches the value in PCR-11. If they match then all is well and the systemd service exits with success. If not, it exits with a fail.

So that's pretty useful. If anything changed, systemd advertizes the failure during boot and dmesg.

### 6.1.3 Hardware

There are presently two versions of hardware that can be used to store signatures on the motherboard. TPM version 1.2 and TPM version 2.0. The major difference between these is that 1.2 chips only support SHA-1, while 2.0 chips support SHA-256. Since (some) signatures now use SHA-256, these are not compatible with TPM 1.2.

To check which is available. For TPM2:

```
bootctl | head -7 | tail -3
```

```
TPM2 Support: yes
Measured UKI: yes
Measured OS: yes
```

whereas for version 1.2 they result shows three *no*'s.

## 6.2 What To Do

Armed with the background on kernel signatures, here are some things that may be helpful.

There are two:

- distinct validation / signatures
- sets of tools to create the keys / signatures
- target systems that validate them.

Table 1: Kernel Signing

| What                | Secure Boot (sbctl --create-keys) | Systemd OS (ukify genkeys)    |
|---------------------|-----------------------------------|-------------------------------|
| does the Validation | UEFI Firmware                     | Systemd / Kernel (OS)         |
| is Validated        | Validates the UKI                 | Validates the state of the OS |

### 6.2.1 Secure Boot and sbctl

The sbctl package must be installed.

To have the kernels signed by local keys, first create the keys:

```
sbctl create-keys
```

You may be prompted to *migrate* an older install, if so then do it:

```
sbctl setup --migrate
```

With the keys in place sbctl will sign the kernel UKI image. Note that if the layout is *bls* then the initrd is separate and unsigned.

You may want to change to UKI where the initrd and the kernel image are packaged into a single file which gets signed. To change the layout please see the [Layout: BLS vs UKI](#) section of the manual

That's all that's required to have a unified kernel image signed by the local keys.

### Signing BLS vs UKI

When it comes to kernel signing, the strongly preferred way is with uki layout. The initrd is part of the uki image that gets signed. In bls layout, the only kernel is signed while neither the initrd nor the kernel command line are signed.

With BLS layout, the kernel signing is handled by the sbctl plugin, */usr/lib/91-sbctl.install*. With UKI layout, ukify handles signing the UKI image.

Best I can tell, the sbctl plugin does not attempt to double sign the image. However, being cautious, sd-boot de-activates the sbctl plugin in this case to eliminate any possibility of a second signature.

### Activating Secure Boot

**Big caveat.** Per the wiki, using local keys can lead to significant problems and may even break the machine.

To use secure boot, the keys must be enrolled and secure boot activated in the UEFI Bios.

To enroll the signing keys the bios needs to have secure boot set to *setup* mode and then boot up the machine and use *sbctl* to enroll the keys.

```
sbctl enroll-keys --microsoft
```

At this point the machine is in secure boot mode and will only boot signed kernels.

### 6.2.2 Systemd-TPM2-Setup

When booting a UKI image with TPM2 hardware where systemd-tpm2-setup has not been configured, a warning is logged:

```
[FAILED] Failed to start Early TPM SRK Setup
```

There are three ways to approach this:

- Ignore it since its benign
- Turn off the systemd service
- Configure the service

## Turn Off the Service

Simply mask the two services:

```
systemctl mask systemd-tpm2-setup-early.service systemd-tpm2-setup.service
```

## Configure the Service

All that's needed to configure this is to generate a key pair and configure ukify to use them to sign the hashed OS components and rebuild the UKI image(s).

This is the correct way to do this:

Configure ukify:

```
cp /usr/lib/kernel/uki.conf /etc/kernel/uki.conf
```

then edit `/etc/kernel/uki.conf` and change `[PCRSignture:NAME]` to `[PCRSignture:all]` and uncomment the three key lines:

```
[PCRSignture:all]
PCRPrivateKey=/etc/systemd/tpm2-pcr-private-key.pem
PCRPublicKey=/etc/systemd/tpm2-pcr-public-key.pem
```

Next, generate the key pair:

```
ukify genkey --config=/etc/kernel/uki.conf
```

Lastly, rebuild the UKI image for each kernel:

```
sd-boot-kernel-update add linux-customr
sd-boot-kernel-update add linux-stable
...
```

You'll see in the output something similar to:

```
/usr/lib/systemd/systemd-measure sign ...
--pcrpkey=/etc/systemd/tpm2-pcr-public-key.pem ...
--private-key=/etc/systemd/tpm2-pcr-private-key.pem
--public-key=/etc/systemd/tpm2-pcr-public-key.pem ...

Signing kernel /boot/EFI/Linux/<machine-id>-<kernel-version>.efi
sd-boot: Completed kernel update linux-custom
```

You can verify this worked on the new UKI image path shown using:

```
# uki='/boot/EFI/Linux/<machine-id>-<kernel-version>.efi'
# ukify inspect "$uki" | grep -E -A2 '\.(pcrpkey|pcrsig)'
.pcrpkey:
  size: 451 bytes
  sha256: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
--
.pcrsig:
  size: 8398 bytes
  sha256: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Of course use the real path of the UKI.

Next time this kernel is booted the the fail log will be replaced with:

Starting TPM SRK Setup...

Please note that while boot times are essentially unchanged, it does take a little longer to build the UKI due to the additional time needed to compute the various hashes.



## DRACUT OPTIONS

sd-boot provides a default dracut config file:

```
/usr/lib/dracut/dracut.conf.d/010-sd-boot-dracut.conf
```

If changes are needed, then overrides can be provided as usual using a drop in file:

```
/etc/dracut.conf.d/xxx.conf
```

since entries found in */etc/* take precedence over */usr/lib/* as usual. Files within each directory are processed in alphanumeric order. For example a file called 020-dracut.conf will override any settings 010-dracut.conf in the same directory. The last file read by dracut sets the options used.

This treatment of configuration file precedence is standard on linux, and kernel-install follows the same rules for it's configuration files.



## EFI FILESYSTEM DRIVERS

While the *EFI* is always Fat-32, a */boot* partition may use other filesystems. In this case the boot loader requires efi filesystem drivers that provided by the *efifs* package.

When this package is installed the driver files are located in the directory */usr/lib/efifs-x86/*, sd-boot detects this, via an alpm hook, and installs them into:

```
<EFI>/EFI/systemd/drivers/
```

where systemd-boot expects to find them.

These can also be manually installed (or removed) from the command line using:

```
/usr/lib/sd-boot/sd-boot-efifs-update add
```

or:

```
/usr/lib/sd-boot/sd-boot-efifs-update remove
```



## SKIPPING KERNEL PLUGINS

kernel-install, by default, calls every plugin. There may be times when there's a need to de-activate a plugin. This is accomplished using a setting in sd-boot config file:

```
/etc/sd-boot/config.yaml
```

Set *skip\_kernel\_plugins* to the list of plugins to be skipped. Each must be a full pathname:

```
skip_kernel_plugins:  
- /usr/lib/kernel/install.d/91-xxx.install  
- /usr/lib/kernel/install.d/91-yyy.install
```

sd-boot effects this by passing the plugin list to be used to kernel-install using the environment variable, `KERNEL_INSTALL_PLUGINS`.

This environment variable contains the list of all plugins to be invoked by kernel-install. The list excludes any of those configured as above to be skipped.



## BOOTABLE EFI TOOLS

Please ensure the Arch package name is listed in `/etc/sd-boot/efi-tools` to let sd-boot know which efi tools it is permitted to install.

sd-boot supports bootable efi tools such as *efi-shell* provided by the *edk2-shell* package.

The alpm hook *99-sd-boot-efi-tool-install.hook* provides the trigger based on package name and the location of the efi file itself is found in

```
/etc/sd-boot/edk2-shell.image
```

This file contains the efi file path. It may also contain comments (lines starting with #).

Loader entries for efi tools are not kernels, and sd-boot uses a kernel-install plugin to modify the entry appropriately.

To add any efi tool and have it *just work*, 2 things are needed.

- ALPM hook files to install and remove the tool from \$BOOT.
- A file containing the path to the bootable efi file itself.

Simplest way to add a new efi tool, is to use the efi shell as templates and modify appropriately.

For example, if the efi tool package is called XXX-efi then the alpm hook install file, installed (as usual) in:

```
/usr/share/libalpm/99-XXX-efi-install.hook
```

should contain:

```
[Trigger]
Type = Package
Operation = Install
Operation = Upgrade
Target = XXX-efi

[Action]
Description = sd-boot: installing an efi tool to EFI
When = PostTransaction
Exec = /usr/lib/sd-boot/sd-boot-efi-tool-update add
NeedsTargets
```

The remove file:

```
/usr/share/libalpm/70-XXX-efi-remove.hook
```

should contain:

```
[Trigger]
Type = Package
Operation = Remove
Target = XXX-efi

[Action]
Description = sd-boot: Removing efi tool from ESP
When = PreTransaction
Exec = /usr/lib/sd-boot/sd-boot-efi-tool-update remove
NeedsTargets
```

The last file provides the location of the efi file itself. This should be located in:

```
/etc/sd-boot/XXX-efi.image
```

and contain the path itself:

```
/usr/share/XXX-efi/x64/xxx-tool.efi
```

## 10.1 A note on memtest86 plus

If sd-boot is used to install this please do not use the (current) version of *memtest86+-efi* in the Arch repo. As of this writing, this package installs files directly into */boot* which violates the basic requirements. It is also somewhat out of date.

I may provide a compatible version to the AUR at some point if its helpful. Note that the latest open source version from memtest.org is 8.x. There is also a non-open source (commercial) version 11.x in the AUR that installs to */usr/share*. The latest open source version is v8.10 at this time.

## DEVELOPMENT

### 11.1 C-Code Language Choice

While I was tempted to do this in python, partly because systemd chose to use it for `/usr/lib/kernel/install.d/60-ukify.install`, I decided to use C.

It's important to be able to do initial testing and validation as non-root user and of course without touching any important files in any actual root directory.

While the the C-code version is a bit more work and a little more complicated to create, the benefit, in my view, is that the c-code is much easier to test and debug and keep organized than bash version.

C also has the advantage that the compiled code has almost no dependencies and is highly performant. Way faster than bash or python. For a system utility this is a pretty nice benefit.

### 11.2 Testing C-Code

Testing and development are done using a *Testing* directory that is writable by non-root user. This is where kernels will be installed, loader entries updated and so on. The tools are installed into this tree and are statically linked.

Otherwise the environment is kept clean other than the *PATH* variable since dracut requires it.

This allows all programs to be tested by leveraging kernel-install's ability to work completely in the testing root tree.

The environment variable *SDB\_DEV\_TEST* activates the test usage, allowing testing and debugging to be done completely within the non-root test tree.

There are two test sets provided under *src/c-code/scripts*. Both test sets should be run from the *src* directory.

- `./tests/scripts/run-test-suite`  
This runs the the tools with *root* set to *Testing/\_\_root\_\_*. This is run as part of the build check process. It also runs the tools under *valgrind*.
- `./tests/scripts/static-analysis:`  
It runs static code analysis using *cppcheck* and *clang-tidy*. Note that kernel-install always runs *chown* and since all tests are run as ordinary (non-root) user this leads to some warning messages

landing in the testing logs. These are benign as kernel-install ignores them. They don't happen in production where kernel-install is running as root and chown is permitted.

For each of these tests, the logs save stdout, stderr and the exit status of the tool along with the output from valgrind.

The image and initrd files will be installed in:

```
Testing/__root__/boot/<machine-id>/<kernel-version>/
```

while the loader entry files, for efi tools and kernels in *bls* layout will be in:

```
Testing/__root__/boot/loader/entries/<machine-id>-<kernel-version>.conf
```

You may notice that the loader entries have a longer path to the kernel image and initrd.

This is normal.

When run in a test tree, kernel-install identifies the *mount* point and removes it from the front of the path. In test mode where the test directory is not actually a mount point (such as */boot*) the pathname written to the loader entry will include more elements. This is fine and when run in production the image file will simply be *linux* instead of */long/path/to/linux*. The same is true for the initrd file as well.

## 11.3 Development and Debugging

Debuggable executables can be installed into the test root directory. From the *src* directory:

```
./do-dev-build  
cd tests/Testing  
export SDB_DEV_TEST=true
```

Then executables can now be debugged using, for example:

```
gdb ./__root__/usr/bin/sd-boot-kernel-update
```

And for this executable for example, within gdb use:

```
run add linux
```

Plugins are called by kernel-install but may be run manually as well:

- *./plugin-tests/run-loaderentry-efi* (for *bls* *plugin-tests/run-loaderentry-kernel*)  
Standalone tests of the plugin that modify the raw loader entry files. The scripts run the tests under valgrind, but there is an option to run in the debugger as well.

## 12.1 Compile and Loader Options

Compilers provide options that can enhance the safety of the compiled code. These are the options currently used to build sd-boot.

### Compile Options:

|                                |                                                 |
|--------------------------------|-------------------------------------------------|
| -pipe                          | Speeds up compilation                           |
| -fno-plt                       | Optimization for shared library calls           |
| -Wmissing-prototypes           | Warn for global function missing prototype decl |
| -fvisibility=hidden            | Only exported symbols in shared lib sym table   |
| -fstack-protector-strong       | Protect function calls with stack canaries      |
| -fstack-clash-protection       | Prevents stack-clash exploits                   |
| -ftrivial-auto-var-init=zero   | Eliminate uninitialized stack memory leaks      |
| -fzero-call-used-regs=used-gpr | Wipe general registers before returning         |
| -mshstk                        | Intel Shadow Stack protection                   |
| -fcf-protection                | Control-flow enforcement                        |

### Link Options:

|                             |                                                       |
|-----------------------------|-------------------------------------------------------|
| -Wl,-as-needed              | Limit shared to 'as needed'                           |
| -Wl,-z,relro                | Mark relocation tables read-only                      |
| -Wl,-z,now                  | Force immediate binding (Full RELRO)                  |
| -Wl,-z,noexecstack          | Strictly enforce non-executable stack                 |
| -Wl,-z,pack-relative-relocs | Optimize layout with DT_RELR packing                  |
| -Wl,-z,defs                 | Catch unresolved symbols at link time, not at runtime |

The test suite is also run using code compiled with **-fanalyzer** as well as **-fsanitize=address,undefined** and no warnings or errors are seen.

## 12.2 Potential Todo Items

- When switching layout from bls to uki, previous kernel is not removed since it was installed using bls layout while *kernel-install remove* is now using uki layout. This leaves the previous bls kernel install instead of removing it. Same would be true switching layout from uki back to bls.  
This means, at least for now, that manual intervention is necessary after changing layout to avoid leaving un-needed files. While it is benign they do take disk space.  
For example after changing to uki layout, check `/boot/loader/entries` and `/boot/<machine-id>/` and remove the older kernel(s) that are no longer needed.  
In uki mode there are no loader entry files at all and kernels are installed in `/boot/EFI/Linux` not in `/boot/<machine-id>`.  
Here `/boot` means either `/boot` or `/efi` as appropriate.
- So, it might be good (nice to have) for the code be more helpful with this.

## 13.1 sd-boot-efifs-update

### 13.1.1 Install or remove efi filesystem drivers to ESP

#### SYNOPSIS

`sd-boot-efifs-update add | remove`

#### DESCRIPTION

Install or remove efi filesystem drivers provided by *efifs* package to the *EFI* partition. The driver files are copied from `/usr/lib/efifs-x64`.

#### ARGUMENTS

`add`

Copies the efi drivers provided by the package and installs them to the EFI partition.

`remove`

Remove the drivers from the EFI partition. This does not touch the source files in `/usr/lib/efifs-x64` which are managed by pacman.

#### FILES

`/usr/lib/sd-boot/sd-boot-efifs-update`

Location of the executable.

`<EFI>/EFI/systemd/drivers/`

The efi partition where the drivers are installed. `<EFI>` is the mount point of the ESP partition.

#### EXAMPLES

To install the drivers:

```
# /usr/lib/sd-boot/sd-boot-efifs-update add
```

To remove the drivers:

```
# /usr/lib/sd-boot/sd-boot-efifs-update remove
```

## SEE ALSO

**sd-boot-efi-tool-update**, **sd-boot-kernel-update**, **sd-boot-find-boot-mounts**, **kernel-install**

## 13.2 sd-boot-efi-tool-update

### 13.2.1 Install or remove an efi tool to \$BOOT

#### SYNOPSIS

```
sd-boot-efi-tool-update add | remove | inspect <package_name>
```

#### DESCRIPTION

Install, remove, or inspect a bootable efi tool in the \$BOOT partition using kernel-install. The \$BOOT partition, following the notation used by kernel-install, is usually one of */boot* or */efi*.

The package must be managed by sd-boot with the package name one of those listed in:

```
/etc/sd-boot/efi-tool.packages
```

It takes two arguments. The first argument is the operation:

```
add | remove | inspect
```

followed by the package name that provides the tool.

For installs using *add*, the path of the source **efi** image must be provided in:

```
/etc/sd-boot/<package-name>.image
```

This file may contain shell style comments as well as the image path.

Since these tools are not kernels, they are installed using *BLS* layout, even when the kernel layout is set to *UKI* in */etc/kernel/install.conf*. This ensures that the efi tools are installed to:

```
/boot/<machine-id>/<package_name>/
```

and that boot loader entry files are appropriately created in:

```
/boot/loader/entries/<machine_name-pacmage_name>.conf
```

#### ARGUMENTS

##### Operation:

- **add**  
Install the efi tool to \$BOOT partition

- remove  
Remove the drivers from \$BOOT partition.
- inspect  
Display information about an installed package.

**Package Name**

Can be name of a package or the string “--all--” which means all efi tools managed by sd-boot.

**FILES**

/etc/sd-boot/<package-name>.packages

The package name must be listed here to permit sd-boot to manage the tool.

/etc/sd-boot/<package-name>.image

This file contains the full path of the efi tool provided by the package. This efi image file will be copied to \$BOOT.

\$BOOT/<machine-id>/<package\_name>-<package\_version>

This is the path where the bootable efi tool is installed.

\$BOOT/loader/entries/<machine-id>-<package\_name>-<package\_version>.conf

This is the type #1 boot loader entry file. It has the title that is used in boot menu along with the path to the program to be booted.

**EXAMPLES**

To install the efi shell from *edk2-shell* package:

```
# sd-boot-efi-tool-update add edk2-shell
```

where edk2-shell is listed in */etc/sd-boot/edk2-shell.packages* and the pathname is provided in the file */etc/sd-boot/edk2-shell.image* which contains:

```
/usr/share/edk2-shell/x64/Shell_Full.efi
```

The image itself is provided by the *edk2-shell* package.

To update all efi tools currently managed by sd-boot:

```
sd-boot-efi-tool-update add "--all--"
```

**13.3 sd-boot-find-boot-mounts****13.3.1 Display ESP and XBOOTLDR partition mount points****SYNOPSIS**

sd-boot-find-boot-mounts

## DESCRIPTION

Displays the list of all ESP and XBOOTLDR partitions. Each is also marked indicating any that are *current* and the *active* ESP used to boot the machine.

Each may have a mark indicating whether that partition is the current one. The ESP used by the motherboard to boot the machine is also saved to nvram. This partition, the one active at boot time, is also marked.

When machine boots the boot loader on the motherboard boots from an ESP and it saves that *active* ESP partition into nvram. The *current* or expected ESP is the one that is designated by the boot order.

Under normal circumstances and almost always, *current* and *active* are the same. See ‘Quirky Esp’ section that discusses the situation when these differ and a possible resolution.

## ARGUMENTS

none

## EXAMPLES

- Example 1

One ESP:

| Device         | Mount | (c) | (a) | type |
|----------------|-------|-----|-----|------|
| /dev/nvme0n1p1 | /boot | (✓) | (✓) | EFI  |

- Example 2:

One ESP and one XBOOTLDR:

| Device    | Mount | (c) | (a) | type     |
|-----------|-------|-----|-----|----------|
| /dev/sda1 | /efi  | (✓) | (✓) | EFI      |
| /dev/sda5 | /boot | (✓) | (✓) | XBOOTLDR |

and the same when run as non-root user:

| Device    | Mount | (c) | (a) | type     |
|-----------|-------|-----|-----|----------|
| /dev/sda1 | /efi  | (✓) | (✓) | EFI      |
| /dev/sda5 | /boot | ( ) | (✓) | XBOOTLDR |

- Example 3

Two ESP and two XBOOTLDR:

| Device         | Mount           | (c) | (a) | type     |
|----------------|-----------------|-----|-----|----------|
| /dev/nvme0n1p1 | /efi            | (✓) | (✓) | EFI      |
| /dev/sdg1      | /mnt/root1/efi  | ( ) | ( ) | EFI      |
| /dev/nvme0n1p2 | /boot           | (✓) | (✓) | XBOOTLDR |
| /dev/sdg5      | /mnt/root1/boot | ( ) | ( ) | XBOOTLDR |

When run as non-root user, any current XBOOTLDR remains unknown/unmarked.

## QUIRKY ESP IN NVRAM

It is possible that the motherboard firmware chooses a different ESP than that defined by nvram boot order. When this happens these 2 ESP partition do not match.

Running *bootctl* when they are mismatched will display:

```
WARNING: The boot loader reports a different partition UUID than the detected ESP.
```

We designate partitions determined by boot nvram as *active* and expected partitions as *current*.

Please note that root privileges are required to check for a current XBOOTLDR partitions. No elevated privileges are otherwise needed.

To manually check for any current ESP and XBOOTLDR partitions:

```
bootctl | grep -E 'ESP:|XBOOTLDR:'
```

And to list all active ESP and XBOOTLDR partitions:

```
fdisk -l | grep -E 'EFI System|Linux extended boot'
```

Also useful to see boot order:

```
efibootmgr
```

and the UUID and PartUUID:

```
lsblk -o NAME,FSTYPE,MOUNTPOINTS,PARTUUID,UUID
```

- Example 4

Two ESP with mismatched active and current

| Device         | Mount          | (c a) type |
|----------------|----------------|------------|
| /dev/nvme0n1p1 | /efi           | (✓ ) EFI   |
| /dev/nvme1n1p1 | /man/disk2/efi | ( ✓) EFI   |

After everything was done to ensure everything being correct and this *quirk* did not go away we found that removing the offending nvram variable resolved the issue. After removing and rebooting the nvram was then updated to the correct value by the motherboard.

To remove the nvram holding the wrong value:

```
cd /sys/firmware/efi/efivars/
echo "Should show the wrong value"
cat LoaderDevicePartUUID-4a67b082-0a4c-41cf-b6c7-440b29bb8c4f

chattr -i LoaderDevicePartUUID-4a67b082-0a4c-41cf-b6c7-440b29bb8c4f
rm LoaderDevicePartUUID-4a67b082-0a4c-41cf-b6c7-440b29bb8c4f
```

Do this at your own risk!

## SEE ALSO

**sd-boot-efi-tool-update**, **sd-boot-kernel-update**, **sd-boot-find-boot-mounts**, **kernel-install**

## 13.4 sd-boot-kernel-update

### 13.4.1 Install (or remove) a kernel to (or from) boot partition

#### SYNOPSIS

```
sd-boot-kernel-update add | remove | inspect <package_name>
```

#### DESCRIPTION

Install, remove or inspect a linux kernel in the \$BOOT partition using kernel-install. The \$BOOT partition, following the notation used by kernel-install, is usually one of */boot* or */efi*.

It takes two arguments. The first argument is the operation:

```
add | remove | inspect
```

followed by the package name providing the kernel.

The kernel package name must be one that is listed to be managed by sd-boot and the kernel package must provide a package basename file that has the package name.

When a kernel package is installed using pacman, it must be installed to:

```
/usr/lib/modules/<kernel_version>
```

It must never install any files to */boot* (or */efi*). kernel-install (and hence sd-boot) are responsible for using vmlinux from */usr/lib/modules/<vers>*, generating the initrd and installing them into \$BOOT.

The layout, specified in */etc/kernel/install.conf* may be set to *bls* or *uki*. In *bls* layout, the kernel and the initrd are installed separately. In addition type #1 boot loader entry files are generated.

In *uki* layout the kernel and initrd are combined into a single unified kernel image (*uki*) and installed into the *efi*. These use type #2 loader entries that are taken directly from the *uki* install and do not use a separate loader entry file,

*uki* layout is preferable. It is simpler, does not require separate loader entry files, and both kernel and initrd are signed since they are in one file. The *uki* file itself is an *efi* file and can therefore be directly booted without the need of a boot manager should that ever be needed.

sd-boot will only act if 2 conditions are met.

- the kernel package name is listed to be managed by sd-boot
- there must be a special file installed by the package in the */usr/lib/modules/<kernel-version>* directory called *pkgbase-sdb* or *pkgbase*. This file should contain the package name.

There is an related tool (*sd-boot-kernel-update-triggers*) used by pacman that is called via an ALPM hook.

It is similar but takes a single argument only (add or remove) and reads the *triggers* from stdin as provided by pacman.

## ARGUMENTS

### Operation

- add <kernel\_package\_name>  
Generate initrd and install the kernel provided in the package <kernel\_package\_name> along with its initrd to \$BOOT
- remove  
Remove the kernel from \$BOOT.
- inspect  
Display information about an installed package.

### Package Name

Can be a kernel package name or the string “-all-” which means all kernels managed by sd-boot.

Note that *add* and *remove* updates files in \$BOOT. pacman is responsible for installing or removing the package itself.

## FILES

/etc/kerne/install.conf

The kernel-install configuration file. The default version provided by sd-boot uses uki layout and dracut for the initrd generator:

```
layout=uki
initrd_generator=dracut
uki_generator=ukify
```

/etc/sd-boot/kernel.packages

The kernel package name must be listed here to permit sd-boot to manage installing or removing the kernel

/usr/lib/modules/<kernel-version>/vmlinuz

This is the kernel, provided by the package, that will be installed to \$BOOT.

/usr/lib/modules/<kernel-version>/pkgbase-sdb

This file contains the name of the kernel package. Arch kernels provide the file *pkgbase*.

While *sd-boot* uses either of these files, *pkgbase-sdb* is preferred since the Arch kernel install tools act on any kernel using *pkgbase*. Using a different filename prevents the Arch tools from installing kernels sd-boot is already handling.

\$BOOT/<machine-id>/<package\_name>-<package\_version>

When layout is *bfs* only, Kernels and initrd files are installed to this directory.

\$BOOT/loader/entries/<machine-id>-<package\_name>-<package\_version>.conf

When layout is *b/s* only, This is the type #1 boot loader entry file. It has the title that is used in boot menu along with the path to the program to be booted.

```
$BOOT/EFI/Linux/<machine-id>-<kernel_version>.efi
```

When layout is *uki* only, the unified kernel image is saved here.

## EXAMPLES

To install the kernel provided by the package *linux-custom*:

```
# sd-boot-kernel-update add linux-custom
```

where *linux-custom* must be listed in the */etc/sd-boot/kernel.packages*.

In addition the *linux-custom* package must provide the file:

```
/usr/lib/modules/<kernel-version>/pkgbase
```

or:

```
/usr/lib/modules/<kernel-version>/pkgbase-sdb
```

that contains one line:

```
linux-custom
```

To inspect all kernels managed by *sd-boot*:

```
sd-boot-kernel-update inspect "--all--"
```

To update all kernels managed by *sd-boot*:

```
sd-boot-kernel-update add "--all--"
```